

7COSC013W: Foundations of AI - Coursework 1
Technical Report

Forecast-Driven Vehicle Routing for Bike-Share Rebalancing
(A Case Study of the Jersey City & Hoboken Citi Bike Network)

Name : Sabesan Thuvaragan

UoW ID : w2223880

IIT ID : 20251067

Course : MSc in Applied Artificial Intelligence

Table of Contents

1. Introduction	4
1.1. What is a Bike-sharing System?	4
1.2. What is Citi Bike?	4
1.3. Problem Statement	5
2. Literature Review	7
2.1. Existing Work	7
3. Dataset Collection	9
3.1. Preparation for Analysis	10
4. Methodology	12
4.1. Why a Hybrid Approach?	12
4.2. Overall Research Approach	12
4.3. Requirements and Constraints	13
5. System Design/ Architecture	14
6. Implementation	16
6.1. Data Loading, Cleaning, and Scoping	16
6.2. Forecasting (Machine Learning)	16
6.3. Priority Scoring	17
6.4. Routing (Search / Optimization)	17
7. Evaluation and Results	19
7.1. Forecasting	19
7.2. Routing	20
7.3. Additional Testing	21
7.3.1. Data File Inspection Summary Comparison	22
7.3.2. Average Bike Net Flow By Hour (Weekday vs. Weekend) Comparison	23

7.3.3.	Station Imbalance Variability (Standard Deviation of Bike Net Flow) Comparison	25
7.3.4.	Error Score (MAE & RMSE) Comparison	26
7.3.5.	Forecast Model Winner Selection Comparison	27
7.3.6.	Routing Model (Total Priority Addressed & Total Distance Travelled) Comparison	28
7.3.7.	Routing Method Winner Selection Comparison	29
8.	Critical Discussion	31
9.	Limitations and Conclusion	32
10.	References	33

Table of Figures

Figure 1: Citi Bike Rental Pricing Chart.....	5
Figure 2: Citi Bike Dataset Values.....	10
Figure 3: System Design/ Architecture.....	14
Figure 4: Forecasting Error Scores of MAE & RMSE of all 4 Methods	19
Figure 5: Forecasting Winner Model Selection	19
Figure 6: Per-station MAE Comparison between all 4 Forecasting Methods.....	20
Figure 7: Per-station RMSE Comparison between all 4 Forecasting Methods.....	20
Figure 8: Routing Winner Model Selection	21
Figure 9: Data File Inspection Summary Comparison - (Additional Testing)	23
Figure 10: Average Bike Net Flow By Hour (Weekday vs. Weekend) Comparison - (Additional Testing)	24
Figure 11: Station Imbalance Variability (Standard Deviation of Bike Net Flow) Comparison - (Additional Testing)	26
Figure 12: Error Score (MAE & RMSE) Comparison - (Additional Testing)	27
Figure 13: Forecast Model Winner Selection Comparison - (Additional Testing).....	28
Figure 14: Routing Model (Total Priority Addressed & Total Distance Travelled) Comparison - (Additional Testing)	29
Figure 15: Routing Method Winner Selection Comparison - (Additional Testing)	30

1. Introduction

1.1. What is a Bike-sharing System?

A bike-sharing system is a public transportation service that allows users to borrow bicycles for short rides around the city. Present-day bicycle-sharing systems are typically either docked networks, in which bicycles are rented and returned to fixed stations, or dock-less systems, in which mobile technology and GPS allow for more flexible pick-up and drop-off. These systems have become an essential aspect of urban transportation, allowing users to utilize public bicycle transport without having to own a bike.

1.2. What is Citi Bike?

Citi Bike, one of the largest such bikeshare systems in the United States, started in 2013, is an excellent illustration of how effective this system can be while also highlighting the challenges it has. It is the nation's largest bikeshare program, with 35,000 bikes and over 2,300 stations throughout Manhattan, Brooklyn, Queens, the Bronx, Jersey City, and Hoboken (Quirk, 2023). Citi Bike charges users in a couple of ways, including per ride (\$4.99 / 30 minutes), day passes (\$25 / day), and memberships (Citi Bike at \$239/year and Lyft Pink at \$199/year), and riders can use any of these options to meet their needs. These prices are subject to change from time to time, and the amounts indicated are the most recent as of July 2026.

Citi Bike even includes a feature called "Bike Angels", in which riders, referred to as Bike Angels, ride bikes from packed stations to those that are running low on bikes, improving the system for everyone. However, Bike Angels is not available to all users. Casual riders who use single rides/ short-term passes are not eligible to participate. This is a special offer which is only available to annual Citi Bike and Lyft Pink members, and in return, those rebalance rides will result in earning points in their membership. So, this scenario alone is enough to cause an imbalance of bikes among stations, and because not all riders are annual members of the system, external rebalancing work on these bikes will very certainly be required, employing vehicles such as vans and trucks to fulfill a proper rebalancing work.

	Single Ride	Day Pass	Citi Bike	Lyft Pink
	\$4.99 for 30 mins	\$25/day	\$239/year	\$199/year
	Get the app →	Get a day pass →	Join →	Join →
Bike unlocks	\$4.99	free	free	free
Ebike prices*	\$0.41/min	\$0.41/min	\$0.27/min	\$0.27/min
Ride times	30 min	30 min	45 min	45 min
Bike Angels			●	●
Rideshare benefits				●

Figure 1: Citi Bike Rental Pricing Chart

1.3. Problem Statement

Given that demand is not fairly distributed throughout the day, some stations run out of bikes whereas others full up completely. A residential station might run out every morning as riders leave for work, whereas a station near a transit hub may fill up with bikes that those same people have just left there. When a station is empty, a rider cannot take a bike, and when it is full, a rider cannot return one. Both possibilities represent a direct breakdown in the service at that moment.

This is important for two related reasons. For the rider, it is a major hassle since a trip cannot begin or end at the exact spot where they want the system to function. For the company, it is a nonstop expenditure because someone must physically transfer bikes between stations to correct the imbalance, through rebalancing, which is usually carried out utilizing vehicles such as vans or trucks that redistribute bikes over the network. Many real-world systems address this either through a predetermined timetable, in which those vehicles travel the same route regardless of the day's actual demand, or reactively, responding only after a station has failed. Both techniques have the same fundamental drawback. They either neglect real variation in demand or act only after a rider has already been let down.

This proposed work targets that issue directly for the Citi Bike network in Jersey City and Hoboken. The goal is to design a system that identifies where and when station imbalances are

likely to occur, and then utilize that forecast to plan an efficient path for a rebalancing vehicle, acting in advance of the problem instead of afterwards.

2. Literature Review

A significant quantity of literature review has been completed on Citi Bike studies in recent years, and some or many of them use intriguing methodologies, either independently or by integrating a variety of methods, as this study does. Few of the similar studies have been addressed below.

A study conducted in 2014 identified bike share rebalancing as a relatively new and interesting subject at the time. They stated that these technologies contribute to achieving more sustainable mobility while reducing traffic and pollution generated by automobile transportation. They also said that since the first bike sharing system was built in Amsterdam in 1965, the number of such programs has grown dramatically, with hundreds of systems now running throughout the world (Dell'Amico et al., 2014).

2.1. Existing Work

In a 2017 study, they merged two components: identifying service level requirements at each bike sharing station and planning (near-)optimal vehicle routes to rebalance inventory (Schuijbroek et al., 2017). Since finding provably optimal solutions is practically intractable, they presented a new cluster-first route-second heuristic, in which a polynomial-size Clustering Problem considers both service level feasibility and estimated routing costs. They computed using real-world data from Hubway (Boston, MA) and Capital Bikeshare (Washington, DC) and found that their algorithm outperformed a pure mixed-integer programming formulation and a constraint programming technique.

One study conducted in 2023 analyzed the data provided by City Bike using methodologies such as exploratory analysis (descriptive/statistical analysis of trip data), forecast modeling, and geo-spatial analysis, recommending ways to improve and optimize the Citi Bike sharing system in Jersey City. This study used the same dataset, but before it was updated to the current one, where it contained some other variables that were present back then but are not now, such as trip duration in seconds, bike ID, rider's gender, year of birth, and so on. This study also generated manual variables such as trip ID, age, age group, month, day of week/month/year, hour of the day, date, and distance between stations. The forecast model consists of three primary components: trends, seasonality, and holidays. They developed an equation to provide their combination, which can be seen in detail in the paper (Voddi, 2023). They employed geospatial analysis to incorporate station

locations and geography. They also employed only the top 13 stations for the routing design, with all route optimizations based on the shortest distance between two specified stations.

Another study focused on data analysis and optimization for Citi Bike sharing, specifically addressing the issue of maintaining system balance during peak rush-hour demand as well as rebalancing overnight to prepare the system for rush-hour usage. (O'Mahony & Shmoys, 2015). For rush hour planning, they focused on stations that frequently run out or are full, as well as stations that self-balance. They concentrated mostly on clustering stations based on their observed usage. They have concentrated on different strategies for rebalancing, such as mid-rush rebalancing (in the morning and evening hours), because usage is highly high and unbalanced during these times, and overnight rebalancing. They used both an empirical and heuristic method in this study.

To check more on hybrid approaches taken in this specific bike-share rebalancing issue, some more study was conducted in this research, and are listed below.

In one study published in 2018, they developed a dynamic rebalancing technique that uses historical data to estimate network conditions and act quickly when necessary. They employed Birth-Death Processes to model station occupancy and determine when to redistribute bikes, using graph theory to choose the rebalancing path and stations (Chiariotti et al., 2018). They validated the suggested methodology using data provided by the Citi Bike NYC bike-sharing system. The numerical simulations demonstrated that a dynamic technique capable of adapting to the network's fluctuating nature outperformed rebalancing methods based on a fixed schedule.

Another research in 2022 presented a hybrid method that mixes rebalancing with clustering to overcome mobility difficulty. They conducted a comparative investigation of various methods for predicting weather and non-weather data, including fuzzy C-means clustering, linear regression, decision tree, and random forest classification (Shir et al., 2022). After rebalancing, they handled the uneven distribution among clusters with the above specified prediction models.

3. Dataset Collection

The data utilized in this study is sourced from Citi Bike’s publicly available system data, which Lyft publishes under the NYCBS Data Use Policy, which specifically allows for academic study. The data includes Ride ID, Rideable type, Started at, Ended at, Start station name, Start station ID, End station name, End station ID, Start latitude, Start longitude, End latitude, End Longitude, as well as Member or casual ride. This data has been analyzed to remove trips taken by Citi Bike staff as they manage and inspect the system, trips taken to and from any of their “test” stations (which they made use of more in June and July 2013), and any trips that were less than 60 seconds in length (potentially accidental starts or users attempting to re-dock a bike to ensure its security). For months with more than 1 million travels, trip data is divided into numerous CSVs within a single compressed file. Therefore, the Citi Bike team has completed all of these steps before making the data publicly available for appropriate usage.

There has been an update in their public dataset, and the image below shows the changes they have made:

We publish [downloadable files of Citi Bike trip data](#). The data includes:

- Ride ID
- Rideable type
- Started at
- Ended at
- Start station name
- Start station ID
- End station name
- End station ID
- Start latitude
- Start longitude
- End latitude
- End Longitude
- Member or casual ride

Data format previously:

- Trip Duration (seconds)
- Start Time and Date
- Stop Time and Date
- Start Station Name
- End Station Name
- Station ID
- Station Lat/Long
- Bike ID
- User Type (Customer = 24-hour pass or 3-day pass user; Subscriber = Annual Member)
- Gender (Zero=unknown; 1=male; 2=female)
- Year of Birth

Figure 2: Citi Bike Dataset Values

Also, the Citi Bike data is published separately by sub-network, and this research only uses the Jersey City/Hoboken data files, and not the entire New York City system, which is far larger and was not part of this project's goal.

3.1. Preparation for Analysis

This project focuses on the Jersey City and Hoboken sub-networks only, as mentioned above. Before being used, the raw data was cleaned, where the trips were inspected for duplicates, trimmed to a clean calendar-month window to remove unnecessary boundary trips from neighboring months, and rows with missing station information were removed, resulting in a clean, reliable set of trip records to work on.

Six months of data (January to June 2026) were chosen to maintain a balance between having enough history to capture genuine weekly and seasonal demand patterns while keeping the data reasonable enough to handle, explore, and assess within the project's timeline. Rather than modeling the entire network, the analysis was limited to the twenty highest traffic stations, selected

primarily based on trip volume. This keeps forecasting and routing problems targeted at the stations where imbalance is most likely to occur, avoiding the added complexity of tracking plenty of low-activity stations with little meaningful demand variation. This scope decision was actively tested rather than assumed: picking the top twenty stations by departures alone, arrivals alone, and combined volume all produced the identical set of stations, indicating that the ranking approach did not bias which stations were included. The cleaned up dataset was also fairly complete, with just a small number of trips excluded due to missing or boundary issues, and the top twenty stations represented a significant portion of all network activity, supporting the argument that this subset is truly indicative of where the real rebalancing issue is primarily centered.

4. Methodology

This project has utilized a hybrid approach: a mix of search and learning methods.

4.1. Why a Hybrid Approach?

This project could potentially be performed using one strategy alone. A purely predictive model that estimates demand and then stops, or a solely search-based routing system that operates with fixed, manually specified priorities. Neither would have tackled the real issue effectively on its own.

Forecast accuracy often improves when further historical data is available, allowing models to better recognize recurring seasonal patterns, long-term trends, and fundamental variability. Providing a richer historical context allows the forecasting engine to make more accurate and dependable predictions (IBM, 2026). However, a forecasting model only predicts where imbalance will occur and says nothing about how to react to it under real-world constraints such as limited time and one vehicle for rebalancing.

A route optimization algorithm on the other hand, is a computational approach or mathematical strategy that finds the most effective and optimal path or set of locations for a given task (Puri, 2025). However, a routing algorithm by itself, with no forecast behind it, would have nothing significant to prioritize and would only be responding to the numbers it was given, with no way of predicting where the problem is headed.

Combining machine learning with search and optimization allows each technique to focus on its strengths: learning from previous patterns to forecast imbalance, and then explicitly reasoning over routes, distances, and limitations to act on that prediction. This is why the project takes a mixed Search and Learning approach rather than committing to one strategy exclusively.

4.2. Overall Research Approach

Rather than selecting a single approach for each component and assuming it will perform well, this research has employed a build, test, compare, and iterate methodology throughout. Instead of committing to a single technique upfront and hoping it performed enough, multiple methods were implemented and tested side by side at each level of the system. When a method underperformed

after being properly tested, it was treated as a genuine finding that required further investigation, rather than being quietly dismissed. This means that final design decisions, such as the forecasting model and routing mechanism to utilize, were made based on direct evidence and not on assumptions. The Implementation and Evaluation chapters discuss the specific methodologies used at each stage, as well as how that comparison was carried out.

4.3. Requirements and Constraints

A number of practical criteria shaped the system's design:

- The system has to function without relying on paid or cost-limited external services. This influenced decisions such as employing straight-line distance between stations instead of a live road-routing API to ensure the system's reliability and reproducibility in the absence of external dependencies.
- This analysis is done in a manner to operate consistently from the exact same input data, without depending on any external data other than the actual Citi Bike dataset itself.
- The routing component needs to reflect a genuinely practical scenario instead of an abstract optimization experiment, so a fixed time schedule, a fixed number of stations, and a single vehicle were used to define the problem in what a real rebalancing operation would truly look like.

These constraints, together with the hybrid Search and Learning approach, directly influenced the system design detailed in the following chapter.

5. System Design/ Architecture



Figure 3: System Design/ Architecture

The system diagram is arranged in the form of a six-stage linear pipeline as displayed above. Raw trip data from Citi Bike’s Jersey City and Hoboken areas is first cleaned and filtered down to the top twenty busiest stations for a six-month period (January to June 2026), resulting in a structured, station-by-hour dataset. This then goes into the demand forecasting stage, where four machine learning approaches are trained and tested to anticipate each station’s net bike traffic for an hour. The best-performing forecast is then turned into a priority score for each station, indicating how urgently it requires attention along with which direction (running empty or full), as well as the urgency indicated by the imbalance count. This priority list is used in the routing phase, where

four search-based strategies are compared for planning an efficient vehicle route that addresses the highest-priority stations within a certain time frame.

Both the forecasting and routing stages enter into an assessment stage, in which each is compared to a naive baseline using real performance measurements, and the best-performing approach at each level is chosen automatically rather than assumed upfront. This structure keeps the two AI approach categories, machine learning and search/optimization, functionally separate yet linked in data flow, representing the hybrid Search and Learning design outlined in the Methodology chapter.

6. Implementation

6.1. Data Loading, Cleaning, and Scoping

The data (Citi Bike NYC dataset) was placed in the author's GitHub repository and loaded straight into the system. The months primarily utilized for this study are January to June 2026, but, however the author stored the data from January 2025 to June 2026 (the last 18 months) for subsequent analysis. The system has been designed in such a way that it can be constructed with the start and end months' data when it is fed into the system, rather than being hardcoded with any predetermined month data values, allowing any length of month-based data to be used for study. If more historical data is to be used, it can be easily accessed on the Citi Bike public dataset page, downloaded and saved in the author's GitHub repository, and then fed into the system. The major purpose for keeping it on GitHub was to prevent manual file loading and data loss during the data feeding process.

In terms of data cleaning, the files were first checked for duplicate data and removed. Then they were inspected for any stray boundary trips from neighboring months and cleaned up. The data was then reviewed for missing values and discarded (such as station information). The data was then filtered and scoped to only include the top 20 highest traffic stations following thorough analysis based on trip volume, in order to keep the problem instance manageable and comprehensible, before moving onto the forecast.

6.2. Forecasting (Machine Learning)

The forecasting component estimates each station's net bike flow (departures - arrivals) for a specific hour. The model's signal is primarily driven by two time-based features: the hour of day and the day of the week. Instead of passing them in as plain numbers, they were encoded cyclically utilizing sine and cosine pairs, allowing the model to correctly interpret hour 23 and hour 0 as near together in time rather than as far apart as possible. Also, a weekend flag is included, which is one-hot encoded into individual binary columns, in addition to the other feature sets.

Four methods were trained and compared: a naive baseline predicting the same value as the same hour, day of week, from one week earlier (so in short, it's being told that, whatever this station did last week, at this exact same time, do that again), followed by a Linear Regression model, a

Random Forest model, and an Ensemble averaging the two. The data was split chronologically instead of randomly, with training covering the first 80% of the time period and testing covering the last 20%, since a random split would allow the model to look into the future compared to what it is being tested on, which is illogical. The best-performing approach was then chosen automatically determined by the lowest test-set MAE, instead of being hardcoded, ensuring that the choice represents actual evidence and adjusts as the underlying data evolves.

6.3. Priority Scoring

This phase connects the forecast with the route decision. Each station-hour's expected net flow is turned into a priority score, which is taken as its absolute value, so a station predicted to run empty is just as important as one that is predicted to run full, regardless of direction. The original prediction's sign is maintained distinctively to indicate the type of problem, either risk of running empty or risk of running full, since it's one of the pieces of information the vehicle requires to act. A threshold is then implemented to ensure that only station hours with a substantially high expected imbalance are flagged as requiring a visit, rather than each non-zero prediction, separating small day-to-day noise from genuine operational difficulties that should be addressed.

6.4. Routing (Search / Optimization)

Given a set of flagged stations for an hour, the routing stage carefully plans a single vehicle's (van/truck) visiting order under a time constraint. Four methods were also implemented here: a naive baseline (by simply visiting the flagged stations in whatever order they naturally arrive in, with no sorting, no logic applied at all), followed by a Priority-Greedy strategy (visiting strictly by urgency), followed by a Priority-Weighted Nearest-Neighbor (heuristic balancing urgency against distance at each step), followed by a Distance-only Nearest-Neighbor (heuristic that always travels to the nearest unvisited flagged station, ignoring priority entirely), which all fall under greedy algorithms.

The Haversine formula is used to determine distance between stations, which yields a straight-line distance from GPS coordinates rather than an actual road-network distance. This allows the system to be self-contained and replicable without relying on an external routing API, but it does not account for actual road layouts, one-way streets, or traffic. This is a limitation that is addressed further in the Critical Discussion chapter. Each technique assumes the same van speed, per-stop

service time, and total time budget, and stops when the budget is surpassed, allowing all four to be compared under similar operating conditions.

All the limitations in the implementation phase are included in the Limitations and Conclusion section at the end.

7. Evaluation and Results

7.1. Forecasting

The Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) were used to analyze all four forecasting methods during the same held-out test period. Random Forest had the lowest error (MAE 2.026, RMSE 3.211), followed by the Ensemble (MAE 2.187, RMSE 3.739), and then the naive baseline (MAE 2.460, RMSE 3.676), with Linear Regression performing the worst (MAE 2.481, RMSE 4.433). Random Forest was automatically chosen as the best-performing approach and carried forward to the routing stage.

Method	MAE	RMSE
Naive Baseline	2.460	3.676
Linear Regression	2.481	4.433
Random Forest	2.026	3.211
Ensemble (LR + RF)	2.187	3.739

Figure 4: Forecasting Error Scores of MAE & RMSE of all 4 Methods

Forecasting Model Selection - Winner : Random Forest			
	Candidate Model	Test MAE	
0	Linear Regression	2.481	
1	Random Forest	2.026	
2	Ensemble (LR + RF)	2.187	
This model's predictions will be used as the forecasting input for the Routing (Search component).			

Figure 5: Forecasting Winner Model Selection

Breaking down error by station revealed that it was not uniformly distributed. “Grove St PATH” and “River St & Newark St”, the two busiest stations in the network, consistently had the worst mistakes across all four techniques, but lower-traffic stations were predicted much more precisely.

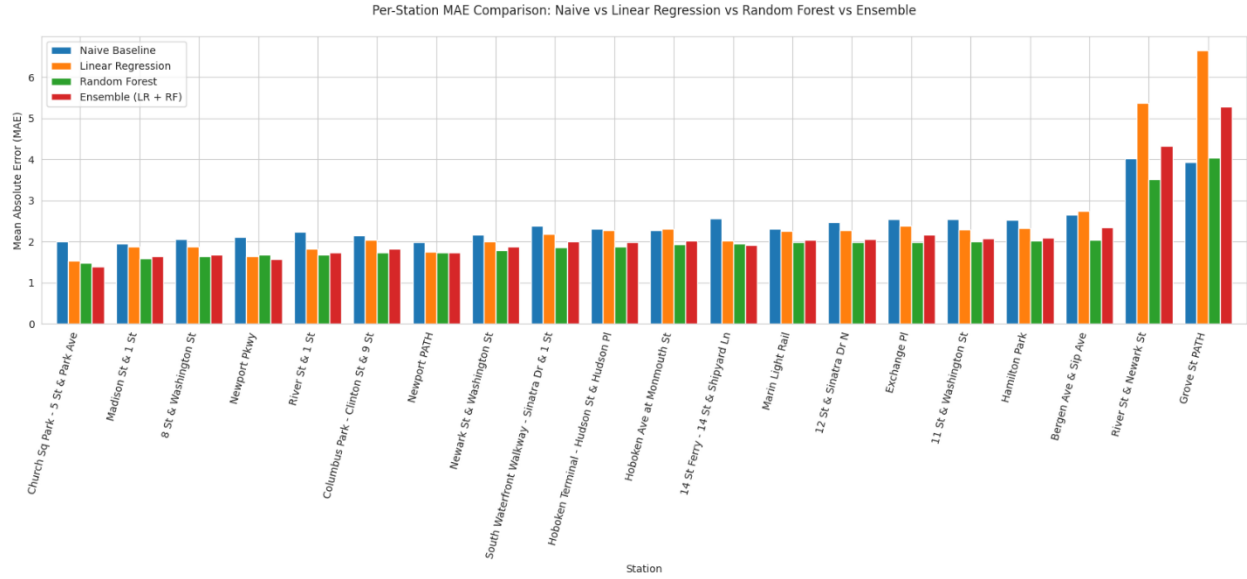


Figure 6: Per-station MAE Comparison between all 4 Forecasting Methods

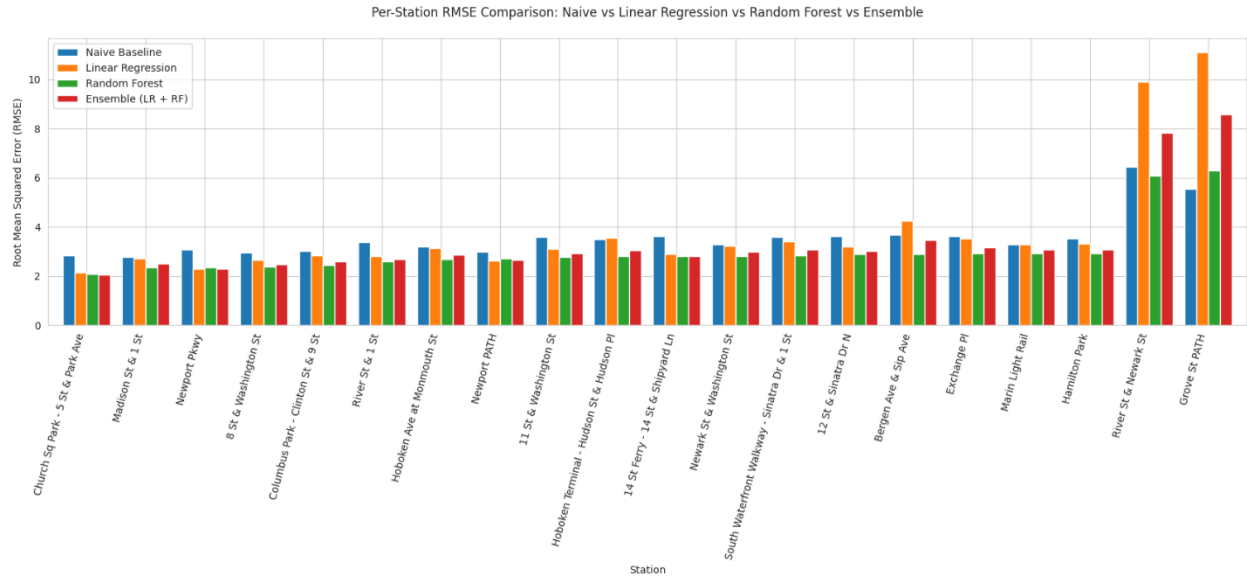


Figure 7: Per-station RMSE Comparison between all 4 Forecasting Methods

7.2. Routing

Every one of the 199 hours throughout the test period with two or more flagged stations was used to test the four routing methods, and the results were combined. The best overall outcome was achieved by distance-only Nearest-Neighbor, which traveled less overall (864.5 km versus 933.9 km for naive) while addressing the same total anticipated priority as the naive baseline (6,390.5).

Priority-Weighted Nearest-Neighbor came in with (6307.9 priority, 1,052.8 km) and Priority-Greedy did the worst on both metrics (6120.0 priority, 1349.3 km). The best-performing approach under the specified selection rule was automatically determined to be Distance-only Nearest-Neighbor.

Final Routing Method Selection - Winner : Distance-only Nearest-Neighbour			
	Routing Method	Total Priority Addressed	Total Distance Travelled (km)
0	Naive Baseline (Raw Order)	6390.5	933.9
1	Priority-Greedy Strategy	6120.0	1349.3
2	Priority-Weighted Nearest-Neighbour	6307.9	1052.8
3	Distance-only Nearest-Neighbour	6390.5	864.5
Total priority addressed : 6390.5			
Total distance travelled : 864.5 km			

Figure 8: Routing Winner Model Selection

To verify this conclusion, Distance-only Nearest-Neighbor routes were compared to the genuine optimal solution, which was discovered by brute-forcing every possible visiting order for 25 hours. It accurately matched the genuine optimal solution in all 25 sampled hours.

7.3. Additional Testing

After the implementation was completed, additional analysis was carried out using two other data periods: 12 months (July 2025 - June 2026), and 18 months (January 2025 - June 2026), respectively. This was done to verify whether the 6-month results were scope dependent, by rerunning the same forecasting and routing pipeline, to examine how the amount of historical data influenced the results.

Different data time periods utilized for testing:

- 6 Months → January 2026 - June 2026
- 12 Months → July 2025 - June 2026
- 18 Months → January 2025 - June 2026

For each proof, the 6, 12, and 18-month timeframe screenshots are grouped together in that order (from top to bottom) for easier viewing, based on the date ranges mentioned above.

7.3.1. Data File Inspection Summary Comparison

Data File Inspection Summary							
Source File	Rows	Date Range Start	Date Range End	Unique Start Stations	Unique End Stations	Rows with Missing Values	
JC-202601-citibike-tripdata.csv	40044	2025-12-31 11:40	2026-01-31 23:47	106	157	2978	
JC-202602-citibike-tripdata.csv	25809	2026-01-31 20:30	2026-02-28 23:53	105	136	1543	
JC-202603-citibike-tripdata.csv	62367	2026-02-28 16:00	2026-03-31 23:46	107	179	1594	
JC-202604-citibike-tripdata.csv	82272	2026-04-01 00:04	2026-04-30 23:59	108	216	1665	
JC-202605-citibike-tripdata.csv	95332	2026-05-01 00:01	2026-05-31 23:53	108	224	1358	
JC-202606-citibike-tripdata.csv	109897	2026-05-31 21:00	2026-06-30 23:55	106	251	1511	

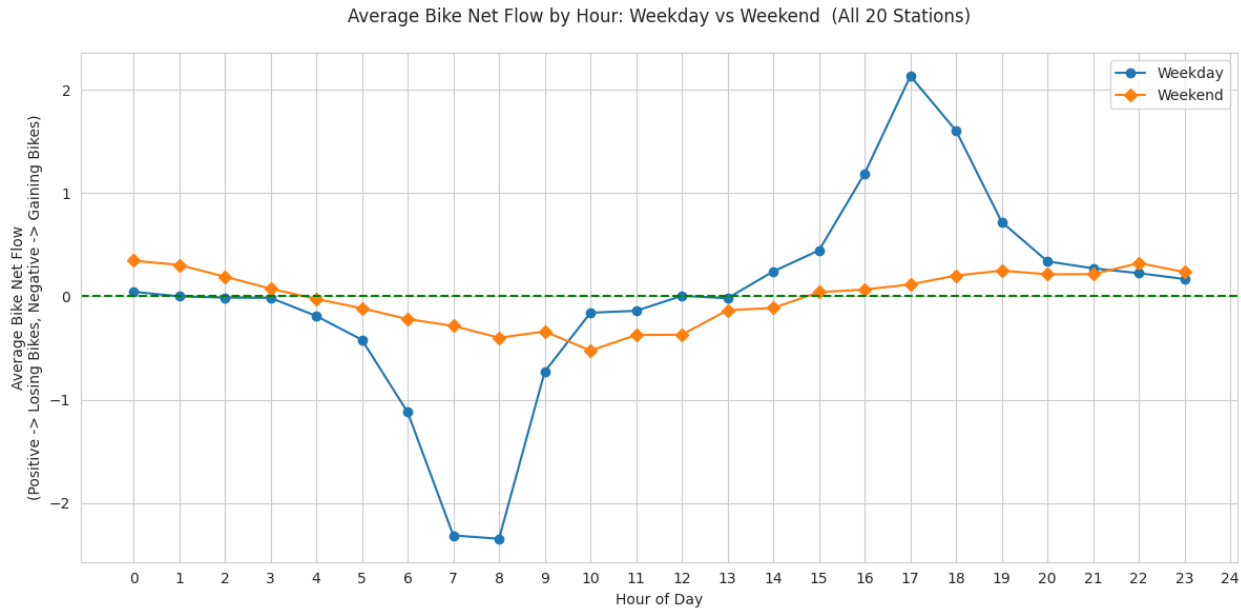
Data File Inspection Summary							
Source File	Rows	Date Range Start	Date Range End	Unique Start Stations	Unique End Stations	Rows with Missing Values	
JC-202507-citibike-tripdata.csv	107726	2025-06-30 16:38	2025-07-31 23:46	84	213	1289	
JC-202508-citibike-tripdata.csv	108441	2025-07-31 23:56	2025-08-31 23:56	88	207	1706	
JC-202509-citibike-tripdata.csv	116071	2025-08-31 09:13	2025-09-30 23:55	94	237	1822	
JC-202510-citibike-tripdata.csv	104205	2025-09-30 08:27	2025-10-31 23:54	94	221	2275	
JC-202511-citibike-tripdata.csv	76724	2025-10-31 15:31	2025-11-30 23:55	94	177	1691	
JC-202512-citibike-tripdata.csv	48474	2025-11-30 23:53	2025-12-31 23:47	108	171	2170	
JC-202601-citibike-tripdata.csv	40044	2025-12-31 11:40	2026-01-31 23:47	106	157	2978	
JC-202602-citibike-tripdata.csv	25809	2026-01-31 20:30	2026-02-28 23:53	105	136	1543	
JC-202603-citibike-tripdata.csv	62367	2026-02-28 16:00	2026-03-31 23:46	107	179	1594	
JC-202604-citibike-tripdata.csv	82272	2026-04-01 00:04	2026-04-30 23:59	108	216	1665	
JC-202605-citibike-tripdata.csv	95332	2026-05-01 00:01	2026-05-31 23:53	108	224	1358	
JC-202606-citibike-tripdata.csv	109897	2026-05-31 21:00	2026-06-30 23:55	106	251	1511	

Data File Inspection Summary						
Source File	Rows	Date Range Start	Date Range End	Unique Start Stations	Unique End Stations	Rows with Missing Values
JC-202501-citibike-tripdata.csv	50611	2024-12-31 23:46	2025-01-31 23:53	85	139	277
JC-202502-citibike-tripdata.csv	45255	2025-01-31 19:03	2025-02-28 23:54	83	126	245
JC-202503-citibike-tripdata.csv	73293	2025-02-28 23:48	2025-03-31 23:32	83	148	337
JC-202504-citibike-tripdata.csv	81553	2025-03-31 22:52	2025-04-30 23:56	83	184	529
JC-202505-citibike-tripdata.csv	93227	2025-04-30 11:59	2025-05-31 23:54	84	182	701
JC-202506-citibike-tripdata.csv	97124	2025-05-31 23:50	2025-06-30 23:50	85	179	1488
JC-202507-citibike-tripdata.csv	107726	2025-06-30 16:38	2025-07-31 23:46	84	213	1289
JC-202508-citibike-tripdata.csv	108441	2025-07-31 23:56	2025-08-31 23:56	88	207	1706
JC-202509-citibike-tripdata.csv	116071	2025-08-31 09:13	2025-09-30 23:55	94	237	1822
JC-202510-citibike-tripdata.csv	104205	2025-09-30 08:27	2025-10-31 23:54	94	221	2275
JC-202511-citibike-tripdata.csv	76724	2025-10-31 15:31	2025-11-30 23:55	94	177	1691
JC-202512-citibike-tripdata.csv	48474	2025-11-30 23:53	2025-12-31 23:47	108	171	2170
JC-202601-citibike-tripdata.csv	40044	2025-12-31 11:40	2026-01-31 23:47	106	157	2978
JC-202602-citibike-tripdata.csv	25809	2026-01-31 20:30	2026-02-28 23:53	105	136	1543
JC-202603-citibike-tripdata.csv	62367	2026-02-28 16:00	2026-03-31 23:46	107	179	1594
JC-202604-citibike-tripdata.csv	82272	2026-04-01 00:04	2026-04-30 23:59	108	216	1665
JC-202605-citibike-tripdata.csv	95332	2026-05-01 00:01	2026-05-31 23:53	108	224	1358
JC-202606-citibike-tripdata.csv	109897	2026-05-31 21:00	2026-06-30 23:55	106	251	1511

Figure 9: Data File Inspection Summary Comparison - (Additional Testing)

7.3.2. Average Bike Net Flow By Hour (Weekday vs. Weekend)

Comparison



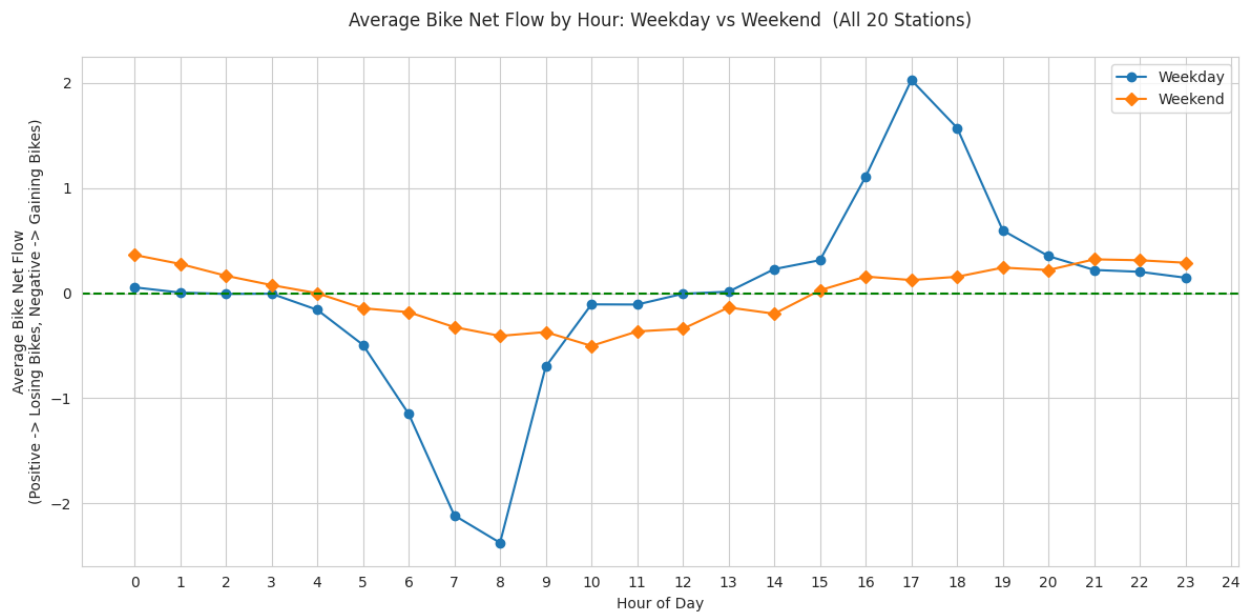
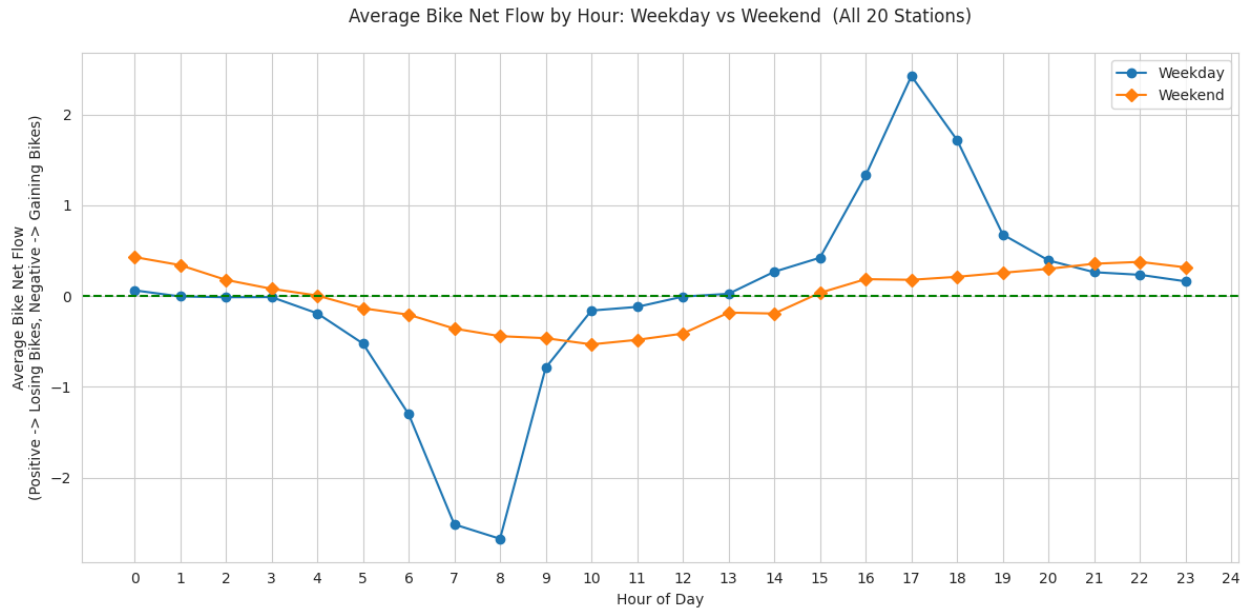
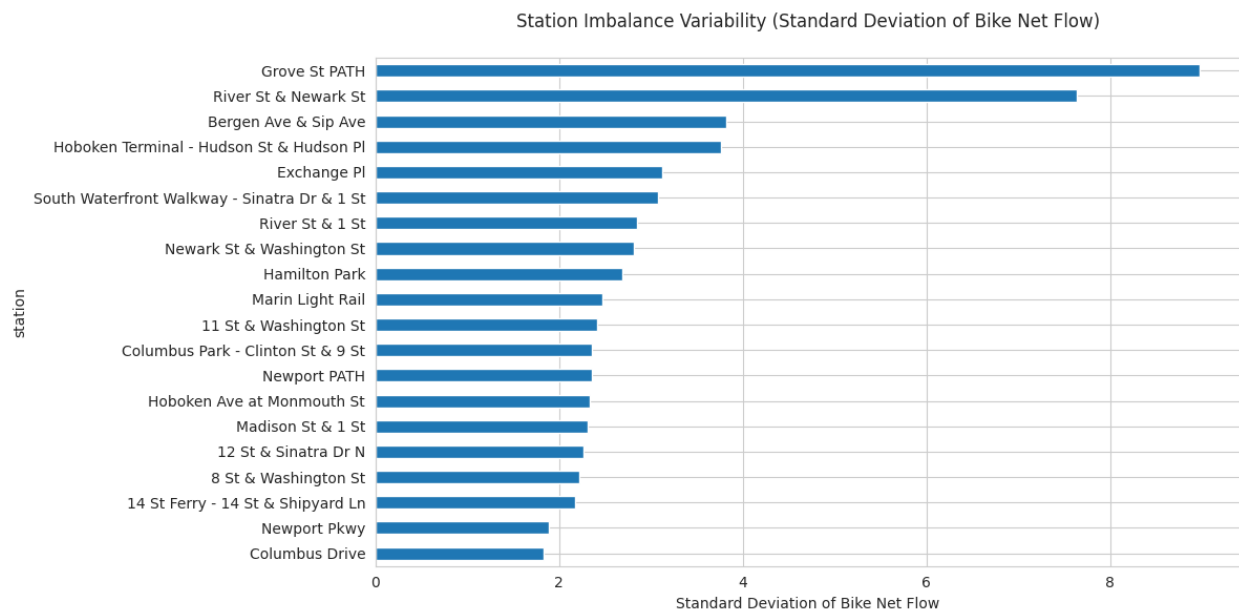
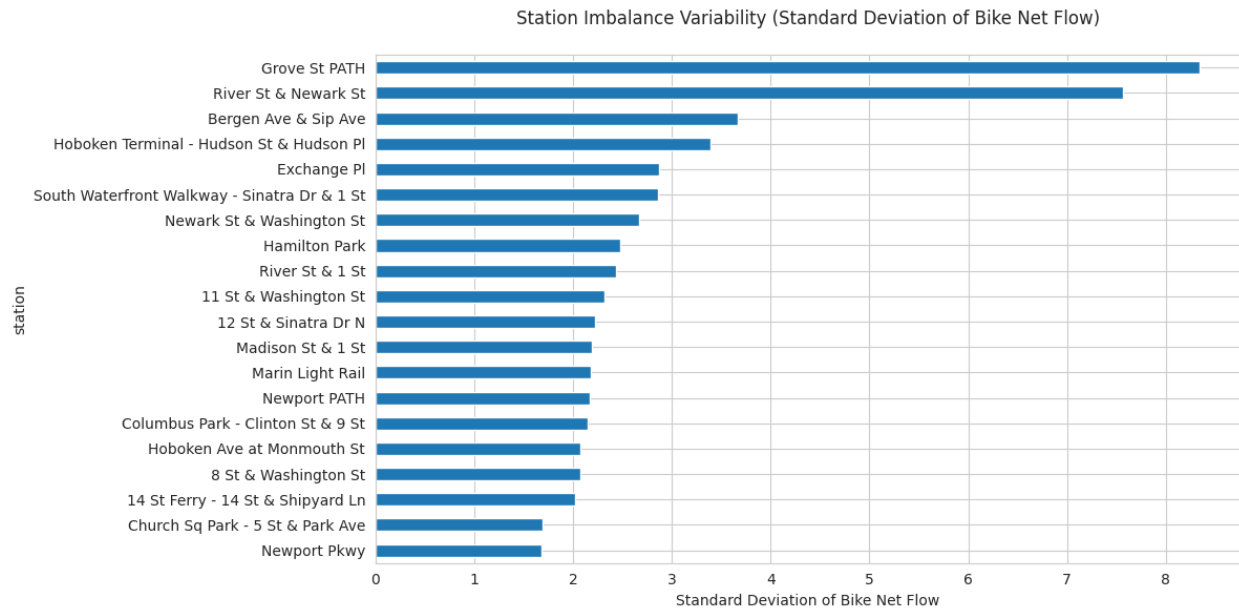


Figure 10: Average Bike Net Flow By Hour (Weekday vs. Weekend) Comparison - (Additional Testing)

7.3.3. Station Imbalance Variability (Standard Deviation of Bike Net Flow) Comparison



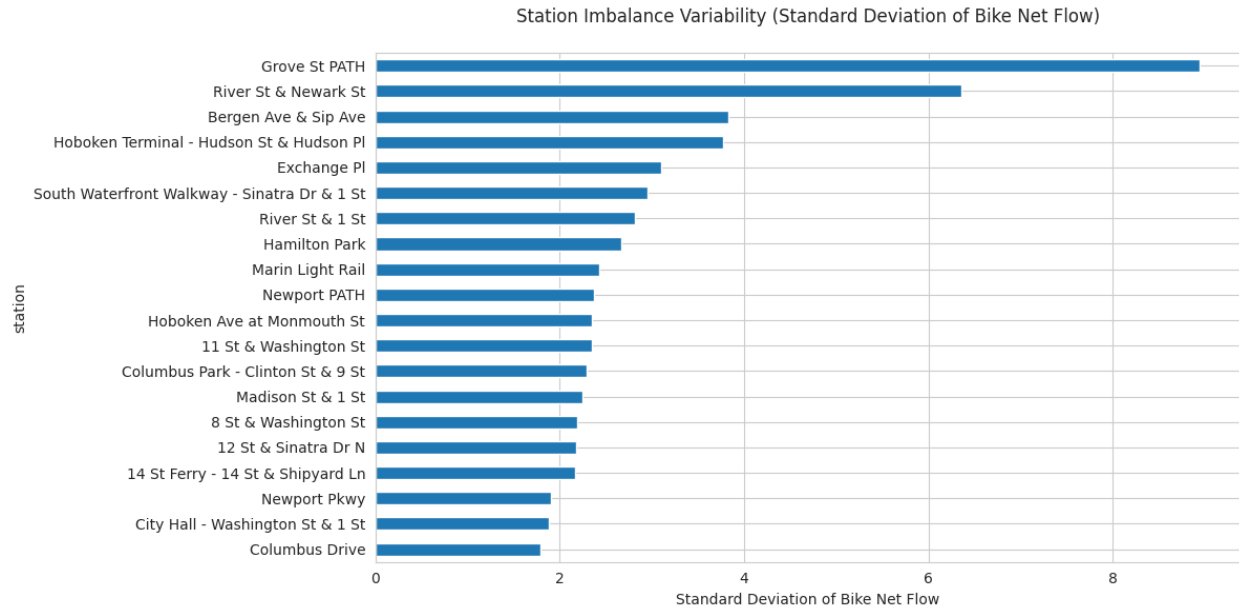
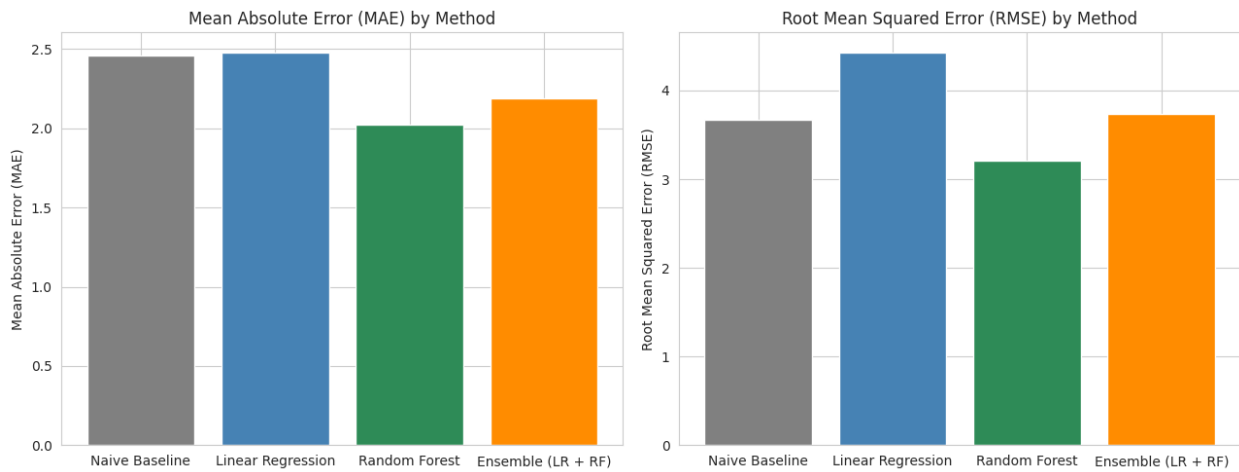


Figure 11: Station Imbalance Variability (Standard Deviation of Bike Net Flow) Comparison - (Additional Testing)

7.3.4. Error Score (MAE & RMSE) Comparison



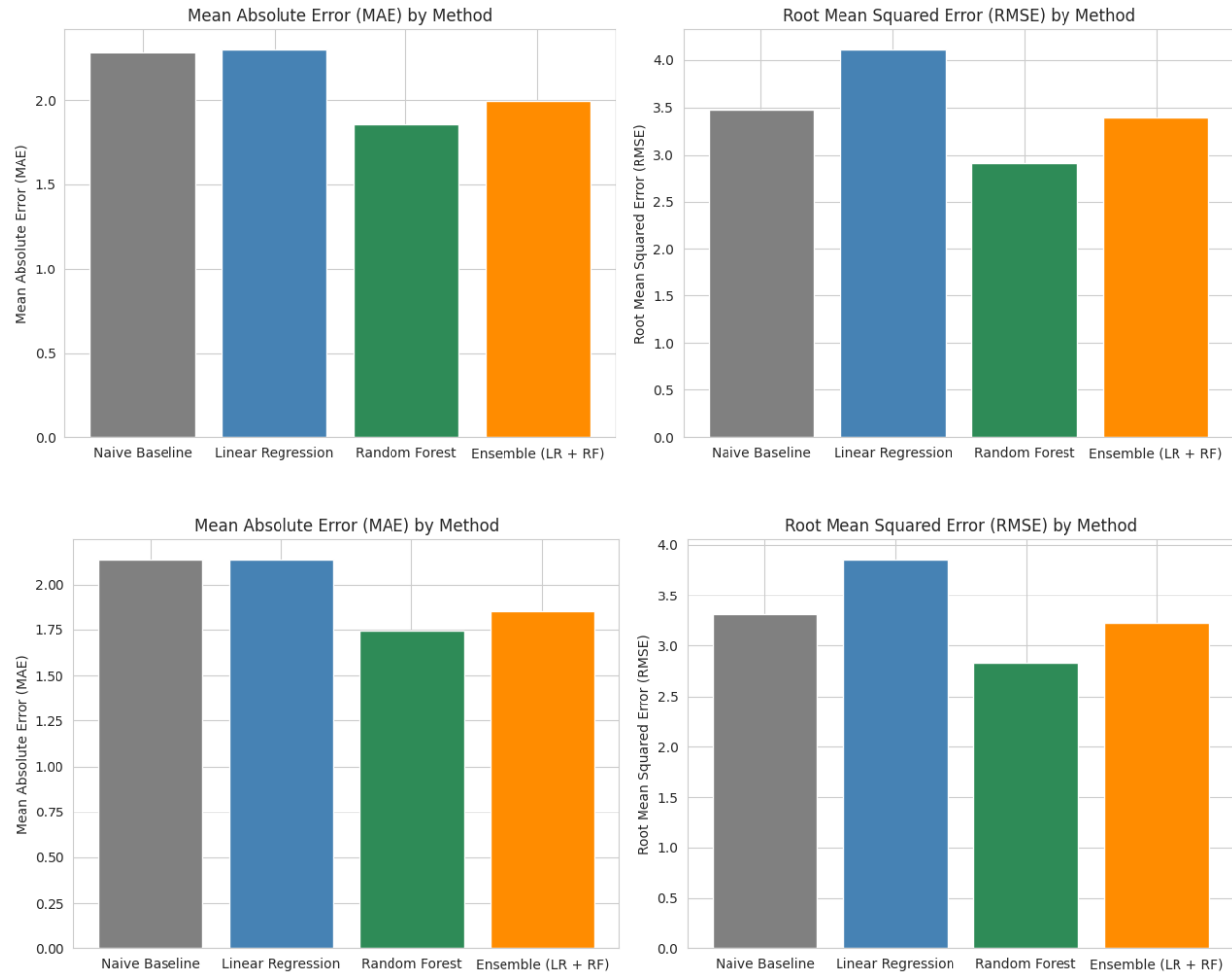


Figure 12: Error Score (MAE & RMSE) Comparison - (Additional Testing)

7.3.5. Forecast Model Winner Selection Comparison

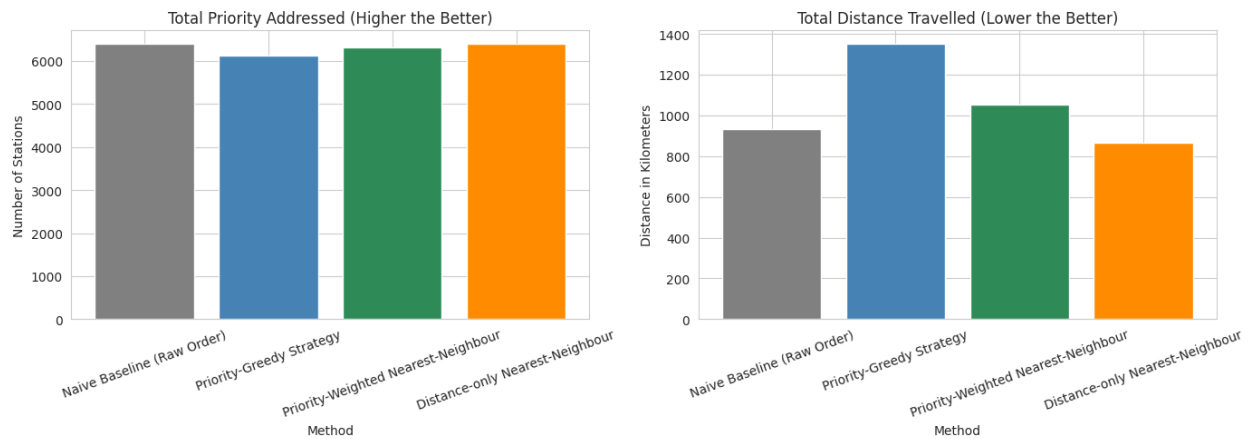
Forecasting Model Selection - Winner : Random Forest		
	Candidate Model	Test MAE
0	Linear Regression	2.481
1	Random Forest	2.026
2	Ensemble (LR + RF)	2.187

Forecasting Model Selection - Winner : Random Forest		
	Candidate Model	Test MAE
0	Linear Regression	2.308
1	Random Forest	1.859
2	Ensemble (LR + RF)	1.996

Forecasting Model Selection - Winner : Random Forest		
	Candidate Model	Test MAE
0	Linear Regression	2.134
1	Random Forest	1.744
2	Ensemble (LR + RF)	1.851

Figure 13: Forecast Model Winner Selection Comparison - (Additional Testing)

7.3.6. Routing Model (Total Priority Addressed & Total Distance Travelled) Comparison



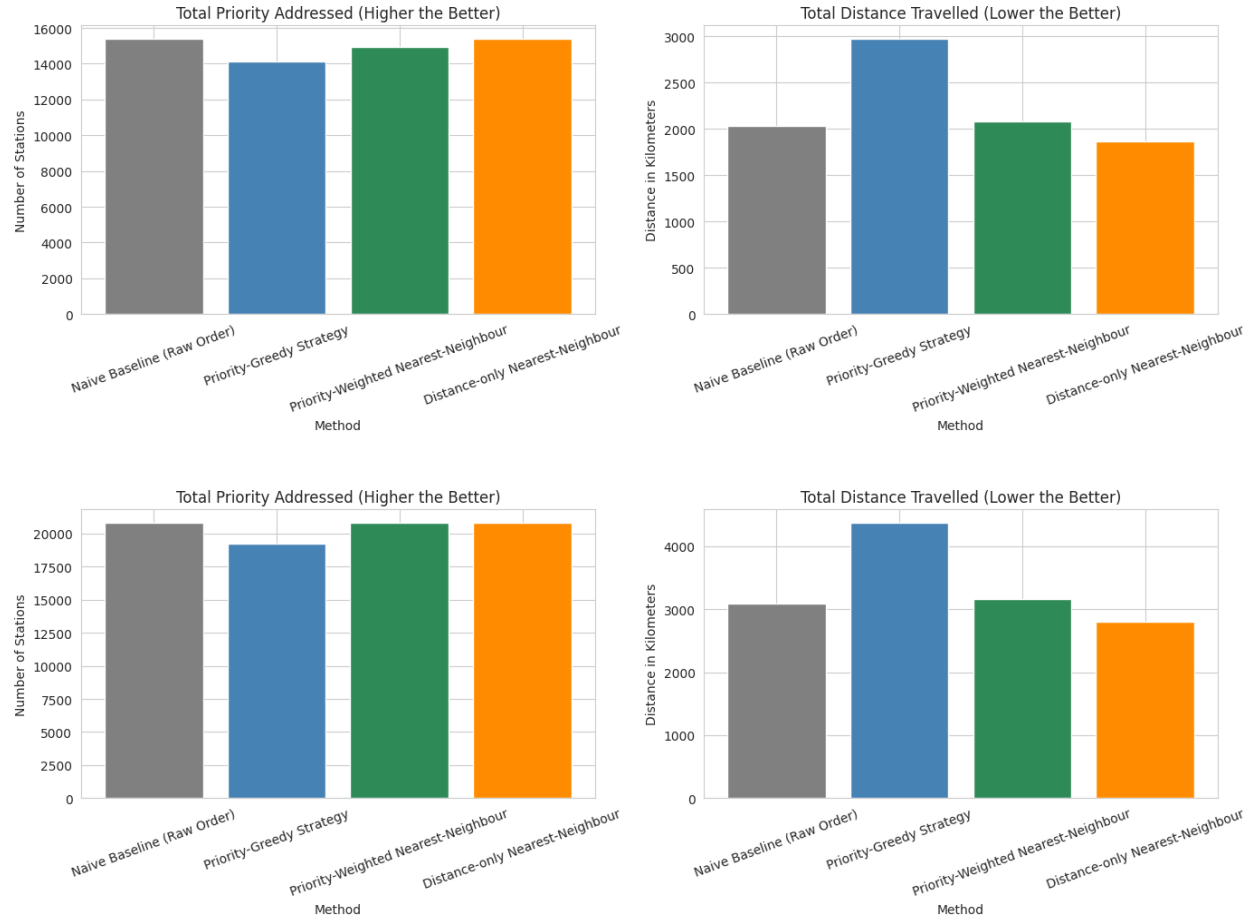


Figure 14: Routing Model (Total Priority Addressed & Total Distance Travelled) Comparison - (Additional Testing)

7.3.7. Routing Method Winner Selection Comparison

Final Routing Method Selection - Winner : Distance-only Nearest-Neighbour			
	Routing Method	Total Priority Addressed	Total Distance Travelled (km)
0	Naive Baseline (Raw Order)	6390.5	933.9
1	Priority-Greedy Strategy	6120.0	1349.3
2	Priority-Weighted Nearest-Neighbour	6307.9	1052.8
3	Distance-only Nearest-Neighbour	6390.5	864.5
Total priority addressed : 6390.5			
Total distance travelled : 864.5 km			

Final Routing Method Selection - Winner : Distance-only Nearest-Neighbour			
	Routing Method	Total Priority Addressed	Total Distance Travelled (km)
0	Naive Baseline (Raw Order)	15369.6	2033.9
1	Priority-Greedy Strategy	14113.4	2967.7
2	Priority-Weighted Nearest-Neighbour	14921.0	2078.4
3	Distance-only Nearest-Neighbour	15369.6	1864.3
Total priority addressed : 15369.6			
Total distance travelled : 1864.3 km			

Final Routing Method Selection - Winner : Distance-only Nearest-Neighbour			
	Routing Method	Total Priority Addressed	Total Distance Travelled (km)
0	Naive Baseline (Raw Order)	20786.3	3092.8
1	Priority-Greedy Strategy	19242.9	4365.6
2	Priority-Weighted Nearest-Neighbour	20786.3	3164.0
3	Distance-only Nearest-Neighbour	20786.3	2796.2
Total priority addressed : 20786.3			
Total distance travelled : 2796.2 km			

Figure 15: Routing Method Winner Selection Comparison - (Additional Testing)

The analysis showed that, regardless of dataset size, the same forecasting and routing/search methods consistently emerged as the best-performing approaches. The only noticeable differences were in the numerical output values, which varied naturally with the amount of data available. This consistency across those three separate timeframes supports the robustness of the chosen approach and suggests that the methodology remains reliable across different data volumes, rather than their success being scoped to the original 6-month timeframe.

Overall, the findings support the authenticity and practical applicability of the research. The results align closely with the project's objectives and indicate that the framework can be confidently applied to varying amounts of data while also providing a strong foundation for future improvements and extensions.

8. Critical Discussion

The forecasting results highlight a genuine, specific weakness in Linear Regression, where, as more data reduced its average error, its RMSE remained higher than the naive baseline's throughout, implying that it got the typical case right more frequently but missed sharp, unpredictable spikes in demand. Random Forest handled these spikes much better since it can bend around unusual patterns rather than putting everything into a single straight line.

In terms of routing, Priority-Greedy prioritized urgency above distance, resulting in routes that made logic one stop at a time but wasted time overall. Priority-Weighted Nearest Neighbor improved on this by taking distance into account, but it still locked in each decision as it proceeded, without considering an earlier decision, which is a known drawback of greedy algorithms in general. Distance-only Nearest-Neighbor won despite completely removing urgency from the decision, demonstrating that for a small, closely packed set of stations, keeping the vehicle's travel time low was more important than chasing the most urgent station first.

The additional testing on 12 and 18-month datasets strengthens both conclusions significantly. Since the same methods won by the identical margins despite how much data was utilized, these are not one-time outcomes connected to a lucky six-month window, but rather consistent, repeating patterns in how each method performs. That repetition is what transforms an interesting outcome into a reliable one, and it applies to both the forecasting and routing aspects of the project.

9. Limitations and Conclusion

This study has some important limitations, which are listed subsequently.

Firstly, this analysis only covers the Jersey City and Hoboken sub-networks and excludes the much larger core New York City Citi Bike system (Manhattan, Brooklyn, Queens, and the Bronx).

The system was narrowed to the twenty highest-traffic stations rather than the entire network of approximately 100 plus, and it used six months of data as its primary timeframe, but longer month timeframes were evaluated for comparison, as previously indicated in the Critical Evaluation section. Random Forest was employed without hyperparameter tuning, and there was no genuine Knowledge Representation component, as the necessity for this research was accomplished purely through Search and Learning hybrid approach.

On the routing aspect, distance was estimated using the Haversine formula rather than true road-network distance, therefore actual street layouts, one-way limitations, and traffic were not considered. The system also assumed a single vehicle (van or truck) with a defined speed and service time, and treated each hour as a separate routing problem, rather than carrying a vehicle's position from one hour to the next. Also, the optimality check was confined to a 25-hour sample rather than the entire test time, due to the computational cost of brute-force searching.

Despite these limitations, the study successfully demonstrated a workable hybrid Search and Learning system that significantly outperforms naive techniques. The forecasting model significantly reduced prediction error, and the routing analysis produced a truly unexpected result: methods which appeared smarter by explicitly prioritizing urgency performed worse than a simpler distance-driven heuristic, which was independently confirmed to be mathematically optimized. This was consistent over three dataset lengths, demonstrating its reliability rather than being a one-time outcome. Overall, the findings are consistent with the project's initial aims and demonstrate that the suggested framework is both realistically useful and a good platform for future work.

10. References

- Chiariotti, F. et al. (2018) A dynamic approach to rebalancing bike-sharing systems, MDPI. Available at: <https://www.mdpi.com/1424-8220/18/2/512> (Accessed: 28 July 2026).
- Citi Bike: Data Sharing Policy (no date) Citi bike: NYC's Official Bike Sharing System. Available at: <https://citibikenyc.com/data-sharing-policy> (Accessed: 28 July 2026).
- Citi bike: NYC's Most Popular Bike Rental Program: Citi bike NYC (no date) Citi Bike: NYC's Official Bike Sharing System. Available at: <https://citibikenyc.com/how-it-works> (Accessed: 29 July 2026).
- Citi Bike: System Data (no date) Citi Bike: NYC's Official Bike Sharing System. Available at: <https://citibikenyc.com/system-data> (Accessed: 28 July 2026).
- Dell'Amico, M. et al. (2014) The bike sharing rebalancing problem: Mathematical formulations and benchmark instances. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0305048313001187> (Accessed: 30 July 2026).
- IBM (2026) Forecasting models. Available at: <https://www.ibm.com/docs/en/apptio-gov/planning-standard/saas?topic=forecasting-models> (Accessed: 29 July 2026).
- O'Mahony, E. and Shmoys, D. (2015) Data analysis and optimization for (Citi)Bike sharing, Proceedings of the AAAI Conference on Artificial Intelligence. Available at: <https://ojs.aaai.org/index.php/AAAI/article/view/9245> (Accessed: 30 July 2026).
- Puri, K. (2025) What is route optimization algorithm? how does it work?, FarEye. Available at: <https://fareye.com/resources/blogs/route-optimization-algorithm> (Accessed: 26 July 2026).
- Quirk, V. (2023) Flood, sweat, and gears: The tumultuous, triumphant history of Citi Bike. Available at: <https://www.lyft.com/blog/posts/flood-sweat-and-gears-the-tumultuous-triumphant-history-of-citi-bike> (Accessed: 1 August 2026).
- Schuijbroek, J., Hampshire, R.C. and Hoeve, W.-J.V. (2017) Inventory rebalancing and vehicle routing in bike sharing systems. Available at:

<https://www.sciencedirect.com/science/article/abs/pii/S0377221716306658> (Accessed: 30 July 2026).

Shir, B., Verma, J.P. and Bhattacharya, P. (2022) Mobility prediction for uneven distribution of bikes in bike sharing systems. Available at: https://www.researchgate.net/publication/365109389_Mobility_prediction_for_uneven_distribution_of_bikes_in_bike_sharing_systems (Accessed: 3 August 2026).

Voddi, V.K.R. (2023) Bike sharing: An in-depth analysis on the Citi Bike Sharing System of Jersey City, NJ | IEEE conference publication | IEEE xplore. Available at: <https://ieeexplore.ieee.org/document/10480792/> (Accessed: 30 July 2026).